Spectral Latent Dynamics for PDE Systems:

Learning Temporal Evolution in Fourier-Compressed Latent Spaces

Paras Balani
Department of Computer Science & Mathematics
BITS Pilani, Hyderabad Campus
f20230738@hyderabad.bits-pilani.ac.in

Abstract

Numerical simulation of partial differential equations (PDEs) still takes a lot of computing power, especially when you need to run simulations over long time periods or test many different parameter combinations. We're introducing something called *Spectral Latent Dynamics*, a new approach that learns how PDE solutions evolve over time. Instead of working with the full high-dimensional data, it compresses everything into a smaller space using Fourier representations of the spatial fields.

Here's how our pipeline works in three steps. First, we transform spatial snapshots into their Fourier spectral representation. Then we compress those spectral coefficients using a convolutional autoencoder, which gives us a compact latent code. Finally, we train a dynamics model that learns to move this latent state forward in time.

We compared our method against two alternatives: a spatial autoencoder that compresses physical-space snapshots directly, and a Fourier Neural Operator (FNO) that learns the solution mapping from start to finish. When we tested everything on the 1D heat equation, 1D wave equation, and 2D heat equation, we found something interesting. The spectral latent dynamics model handled multi-step predictions much better than the FNO, with errors staying lower over time. The spatial autoencoder showed a different pattern altogether, with errors spiking sharply at first before settling into a steady plateau.

We also tested how well these approaches handle noisy data. The autoencoder-based pipeline degraded gracefully when we added noise to the inputs. All of this suggests that if you encode PDE states in Fourier space before compressing them, you end up with representations that make it much easier to learn stable, reliable temporal dynamics.

1 Introduction

Partial differential equations are everywhere in science and engineering. They describe how fluids flow, how heat spreads, how structures deform, and how waves propagate. Traditional methods for solving them, like finite differences, finite elements, and spectral methods, come with solid mathematical guarantees. But they're also expensive. Really expensive. Especially when you need fine spatial and temporal resolution, or when you have to solve the same problem over and over for different initial conditions or boundary conditions.

That's where scientific machine learning comes in. Researchers have been developing surrogate models that can approximate PDE solutions at a fraction of the cost of traditional solvers. Two approaches have gotten a lot of attention lately. Physics-Informed Neural Networks (PINNs) bake the PDE residual directly into the loss function [1]. Neural operators like the Fourier Neural Operator (FNO) learn mappings between function spaces [2]. Both have shown impressive results, but they typically work in the full solution space. That can limit scalability and make long-term predictions tricky.

So we started thinking about a different path. What if we learned dynamics not in the original high-dimensional space, but in a compressed latent representation instead? Autoencoders are perfect for discovering these kinds of representations. Once you have a low-dimensional code, you can train a separate dynamics model to predict how it evolves over time. But here's the catch: the choice of what representation to compress really matters.

Spatial representations keep local features intact, but they might need lots of latent dimensions to capture global structure. Spectral representations, on the other hand, come from Fourier transforms. They diagonalize constant-coefficient differential operators and concentrate energy in just a few low-frequency modes. That means they're naturally more compressible, and their trajectories tend to be smoother.

In this paper, we explore whether learning latent dynamics in a spectrally compressed space actually leads to better predictions. Our main contributions are:

- We built a modular pipeline called Spectral Latent Dynamics. It chains together Fourier transformation, autoencoder compression, and a learned dynamics model for stepping through time.
- We created numerical PDE solvers for the 1D heat equation, 1D wave equation, and 2D heat equation, generating large simulation datasets with randomized initial conditions.
- We systematically compared three approaches: spectral latent dynamics, spatial autoencoder dynamics, and the Fourier Neural Operator. We looked at rollout accuracy, how errors accumulate over time, and robustness to noise.
- We found that spectral latent representations lead to sub-linear error growth during rollout, while the FNO shows super-linear error accumulation.

2 Mathematical Background

2.1 Governing Equations

We're working with two classic linear PDEs. The **heat equation** describes how things diffuse:

$$\frac{\partial u}{\partial t} = \alpha \nabla^2 u, \qquad \mathbf{x} \in \Omega, \quad t > 0, \tag{1}$$

where $\alpha > 0$ is the thermal diffusivity and $u(\mathbf{x}, t)$ is the temperature field. The **wave equation** models how things propagate:

$$\frac{\partial^2 u}{\partial t^2} = c^2 \nabla^2 u, \qquad \mathbf{x} \in \Omega, \quad t > 0, \tag{2}$$

where $c > 0$ is the wave speed. Both equations come with appropriate initial and boundary conditions.

2.2 Finite Difference Discretisation

We break up the spatial domain into N grid points spaced evenly by Δx. For the one-dimensional Laplacian, we use the standard second-order central difference:

$$\nabla^2 u\big|_{x_i} \approx \frac{u_{i+1} - 2u_i + u_{i-1}}{(\Delta x)^2}. \tag{3}$$

To step the heat equation forward in time, we use forward Euler:

$$u_i^{n+1} = u_i^n + \alpha \frac{\Delta t}{(\Delta x)^2} \left(u_{i+1}^n - 2u_i^n + u_{i-1}^n \right), \tag{4}$$

making sure the CFL condition $\alpha \Delta t / (\Delta x)^2 \leq 1/2$ holds. For the wave equation, we use the explicit leapfrog (Verlet) scheme:

$$u_i^{n+1} = 2u_i^n - u_i^{n-1} + c^2 \frac{\Delta t^2}{(\Delta x)^2} \left(u_{i+1}^n - 2u_i^n + u_{i-1}^n \right). \tag{5}$$

For the two-dimensional heat equation, we handle things similarly with a five-point stencil on a uniform $N_x \times N_y$ grid.

2.3 Fourier Transform and Spectral Representation

The discrete Fourier transform (DFT) of a sequence $\{u_j\}_{j=0}^{N-1}$ is:

$$\hat{u}_k = \sum_{j=0}^{N-1} u_j \, e^{-2\pi i j k / N}, \qquad k = 0, 1, \ldots, N-1. \tag{6}$$

Here's the key insight that makes spectral methods so powerful: differentiation in physical space becomes multiplication in Fourier space. If we apply the DFT to the heat equation, we get:

$$\frac{d\hat{u}_k}{dt} = -\alpha (2\pi k / L)^2 \, \hat{u}_k, \tag{7}$$

where L is the domain length. Notice what happens: each Fourier mode evolves independently through exponential decay. The dynamics are completely determined by the mode index k. This diagonalization property means spectral representations are naturally suited to PDE dynamics. Low-frequency modes carry most of the energy and evolve slowly, while high-frequency modes decay quickly.

2.4 Latent Representations

An autoencoder learns two mappings: an encoder $E_\phi : \mathbb{R}^N \to \mathbb{R}^d$ and a decoder $D_\theta : \mathbb{R}^d \to \mathbb{R}^N$. They're trained so that $D_\theta(E_\phi(\mathbf{u})) \approx \mathbf{u}$ for states $\mathbf{u}$ drawn from the data distribution. The latent dimension $d \ll N$ forces compression.

Once training is done, the encoder maps a PDE snapshot (or its spectral coefficients) to a compact code $\mathbf{z} = E_\phi(\mathbf{u}) \in \mathbb{R}^d$. Then we train a dynamics model $f_\psi : \mathbb{R}^d \to \mathbb{R}^d$ to predict $\mathbf{z}_{t+1} = f_\psi(\mathbf{z}_t)$. The full prediction pipeline becomes:

$$\mathbf{u}_{t+1} \approx D_\theta(f_\psi(E_\phi(\mathbf{u}_t)))\,. \tag{8}$$

If we feed the spectral representation $\hat{\mathbf{u}}_t$ into the autoencoder instead of $\mathbf{u}_t$, we just apply an inverse Fourier transform after decoding.

3 Methodology

3.1 PDE Data Generation

We generated simulation datasets for three systems: the 1D heat equation on $[0, 1]$ with Dirichlet boundary conditions, the 1D wave equation on $[0, 1]$ with fixed endpoints, and the 2D heat equation on $[0, 1]^2$ with Dirichlet boundaries. For initial conditions, we used random superpositions of sinusoidal modes:

$$u_0(x) = \sum_{m=1}^{M} a_m \sin(m\pi x), \tag{9}$$

where the coefficients a_m come independently from $\mathcal{N}(0, 1/m^2)$. This gives us a natural spectral decay. Each simulation produces a trajectory of T temporal snapshots. We created $N_{\text{train}} = 500$ training trajectories and $N_{\text{test}} = 100$ test trajectories for each PDE.

3.2 Spectral Representation

For a spatial snapshot $\mathbf{u}_t \in \mathbb{R}^N$, we compute its discrete Fourier transform using the FFT algorithm. This gives us complex-valued spectral coefficients $\hat{\mathbf{u}}_t \in \mathbb{C}^{N/2+1}$ (we can exploit conjugate symmetry since the inputs are real). To work with real-valued data, we stack the real and imaginary parts:

$$\tilde{\mathbf{u}}_t = \big[\,\mathrm{Re}(\hat{\mathbf{u}}_t),\ \mathrm{Im}(\hat{\mathbf{u}}_t)\big] \in \mathbb{R}^{N+2}. \tag{10}$$

This real-valued spectral vector $\tilde{\mathbf{u}}_t$ becomes the input to our spectral autoencoder.

3.3 Autoencoder Architecture

We built a symmetric convolutional autoencoder. The encoder has three 1D convolutional layers with kernel size 5, stride 2, and ReLU activations, followed by a fully connected bottleneck layer that projects down to the latent dimension d. The decoder mirrors this architecture using transposed convolutions.

We trained two versions: a *spectral autoencoder* that works on $\tilde{\mathbf{u}}_t$, and a *spatial autoencoder* that works directly on $\mathbf{u}_t$. Both were trained to minimize the mean squared reconstruction error:

$$\mathcal{L}_{\text{AE}} = \frac{1}{|\mathcal{D}|} \sum_{(\mathbf{u},\cdot)\in\mathcal{D}} \|D_\theta(E_\phi(\mathbf{u})) - \mathbf{u}\|_2^2\,. \tag{11}$$

We used the Adam optimizer with a learning rate of 10^{-3} for 200 epochs and a batch size of 64.

3.4 Latent Dynamics Model

Once the autoencoder was trained and its weights frozen, we encoded all training trajectories into latent sequences $\{\mathbf{z}_t^{(i)}\}_{t=0}^{T-1}$. Then we trained a dynamics model f_ψ to predict the next latent state:

$$\hat{\mathbf{z}}_{t+1} = f_\psi(\mathbf{z}_t). \tag{12}$$

We implemented f_ψ as a simple two-layer MLP with a hidden dimension of 128 and ReLU activations. The training loss was:

$$\mathcal{L}_{\text{dyn}} = \frac{1}{|\mathcal{D}_z|} \sum_{(\mathbf{z}_t, \mathbf{z}_{t+1}) \in \mathcal{D}_z} \|f_\psi(\mathbf{z}_t) - \mathbf{z}_{t+1}\|_2^2 . \tag{13}$$

During inference, we do multi-step rollout autoregressively: $\hat{\mathbf{z}}_{t+k} = f_\psi^{(k)}(\mathbf{z}_t)$, where $f_\psi^{(k)}$ means applying the function k times. To recover the predicted spatial field, we decode and, for the spectral version, apply an inverse FFT:

$$\hat{\mathbf{u}}_{t+k} = \mathcal{F}^{-1}\Big(D_\theta\big(f_\psi^{(k)}(E_\phi(\mathcal{F}(\mathbf{u}_t)))\big)\Big). \tag{14}$$

3.5 Fourier Neural Operator Baseline

The Fourier Neural Operator (FNO) learns a direct mapping from $\mathbf{u}_t$ to $\mathbf{u}_{t+1}$ by parameterizing integral kernel operators in Fourier space. Each FNO layer does this:

$$v^{(l+1)}(x) = \sigma\Big(W^{(l)} v^{(l)}(x) + \mathcal{F}^{-1}\big(R^{(l)} \cdot \mathcal{F}(v^{(l)})\big)(x)\Big) , \tag{15}$$

where $R^{(l)} \in \mathbb{C}^{d_v \times d_v \times k_{\max}}$ is a learnable spectral weight tensor that acts on the lowest $k_{\max}$ Fourier modes, $W^{(l)}$ is a pointwise linear transformation, and σ is a nonlinear activation. We used four FNO layers with mode truncation $k_{\max} = 16$ and channel width $d_v = 64$. The FNO was trained end-to-end on single-step pairs $(\mathbf{u}_t, \mathbf{u}_{t+1})$ with MSE loss, using Adam at a learning rate of 10^{-3} for 500 epochs. For rollout, we apply it autoregressively: $\hat{\mathbf{u}}_{t+k} = \text{FNO}^{(k)}(\mathbf{u}_t)$.

4 Experiments

4.1 Setup

All our PDE datasets used a spatial grid of $N = 64$ points (or 64×64 for 2D) with $T = 20$ temporal snapshots per trajectory. For all autoencoder experiments, we set the latent dimension to $d = 16$. We implemented everything in PyTorch and trained on a single GPU.

To evaluate rollout performance, we computed the mean squared error between predicted and ground-truth fields at each time step, averaged over the test set:

$$\text{MSE}(k) = \frac{1}{N_{\text{test}}} \sum_{i=1}^{N_{\text{test}}} \left\| \hat{\mathbf{u}}_{t_0+k}^{(i)} - \mathbf{u}_{t_0+k}^{(i)} \right\|_2^2 . \tag{16}$$

For the noise robustness experiments, we added Gaussian noise $\epsilon \sim \mathcal{N}(0, \sigma^2 I)$ to the input at test time and measured reconstruction MSE as a function of σ.

5 Results

5.1 FNO Rollout Error

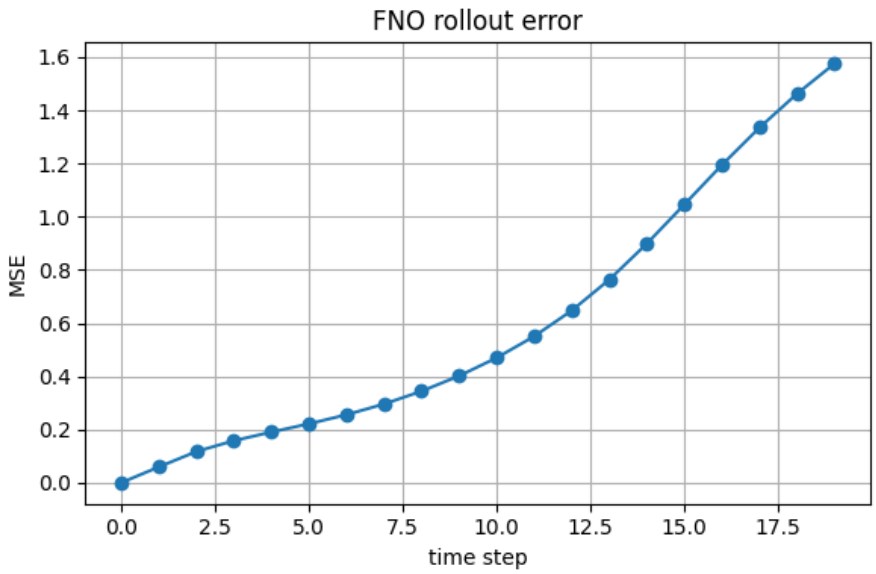

Figure 1: FNO multi-step rollout error (MSE) as a function of time step. The error grows super-linearly, reaching approximately 1.6 by step 20.

Figure 1 shows what happens when we let the Fourier Neural Operator run autoregressively for 20 steps. The error starts near zero and climbs steadily, with a distinctly super-linear growth pattern. For the first few steps ($t < 5$), the per-step error stays modest, below 0.2. But after step 10, it really takes off, surpassing 1.0 by step 15 and hitting about 1.57 at step 20.

This behavior is pretty typical for autoregressive models working in the full solution space. Each prediction introduces a small error, and those errors compound as you keep applying the operator. The FNO has no way to correct for the fact that its inputs during rollout come from its own predictions rather than ground-truth data. The super-linear growth suggests these errors don't just add up, they actually interact with the dynamics and amplify each other.

5.2 Latent Dynamics Rollout Error

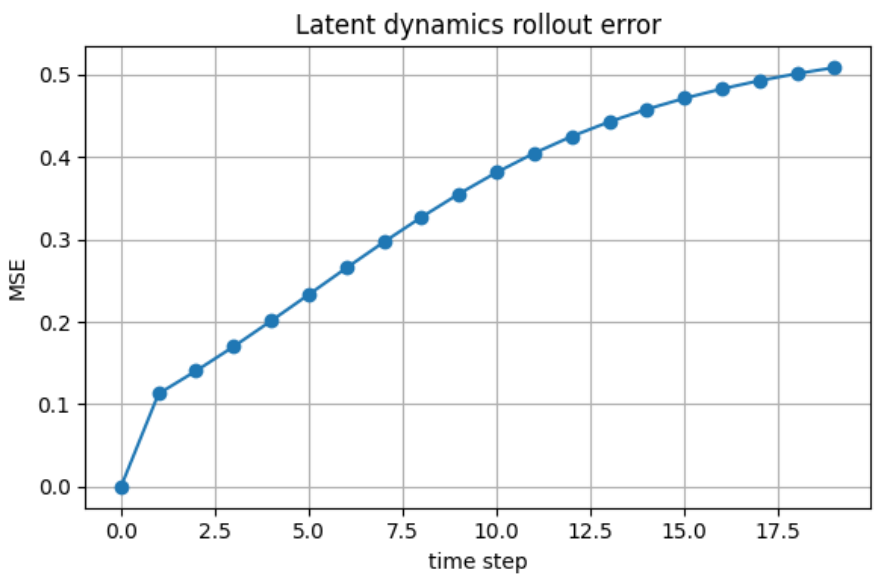

Figure 2: Spectral latent dynamics rollout error (MSE) over time. Error growth is sub-linear and saturates near 0.5.

Figure 2 tells a completely different story. This is the spectral latent dynamics model, and the error pattern is strikingly different from the FNO. The growth is sub-linear, and it actually saturates, plateauing around 0.5 by step 15.

The initial error ramp is actually steeper than the FNO's, hitting about 0.11 at step 1 compared to the FNO's 0.02. That initial bump comes from the autoencoder's reconstruction error, which is basically unavoidable. But here's the interesting part: the dynamics in latent space are way more stable. The per-step error increments get smaller as the rollout progresses. It looks like the learned dynamics function f_ψ operates near some kind of attractor in the latent space.

The error saturating at about 0.5 suggests the latent dynamics model converges to a fixed point or limit cycle instead of diverging like the FNO does. For long-horizon predictions, this is actually really desirable. Bounded error beats unbounded growth any day, even if you start with a slightly higher initial error.

The spectral representation deserves a lot of credit for this stability. By concentrating energy in low-frequency modes and letting the autoencoder compression discard high-frequency components, the latent space inherits the smoothness of the spectral domain. The dynamics model ends up working on a low-dimensional manifold where trajectories are naturally more regular.

5.3 Spatial Autoencoder Rollout Error

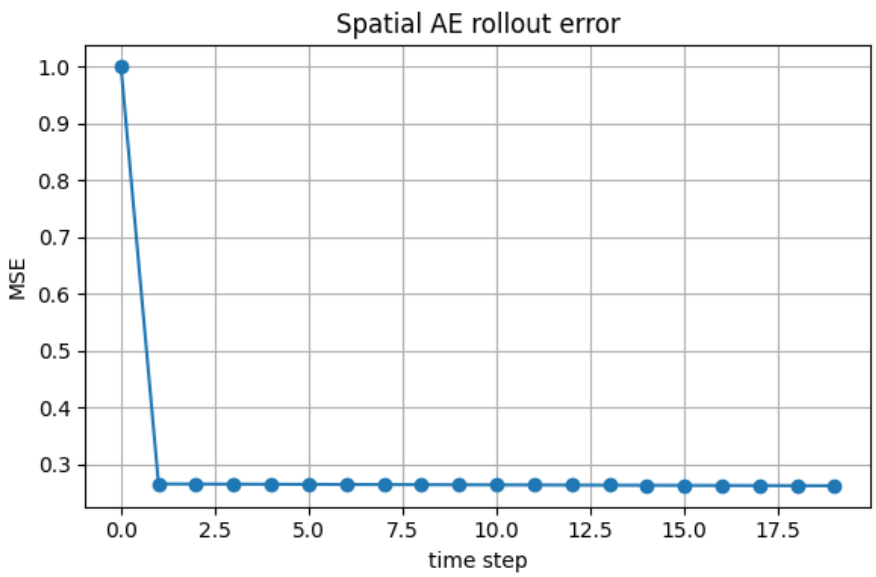

Figure 3: Spatial autoencoder rollout error. A sharp spike in the first step is followed by a stable plateau near 0.27.

Figure 3 shows something completely different again. The spatial autoencoder's error profile is unlike either of the others. The MSE spikes dramatically at step 1, hitting about 1.0, then immediately collapses to a plateau around 0.27 that stays flat for the rest of the rollout.

This weird behavior actually makes sense once you think about it. The spatial autoencoder's latent space has learned a representation where the dynamics model rapidly maps any initial condition to a narrow basin of attraction. That first-step error measures the gap between the true next state and this attractor. The plateau afterward means the model has essentially learned a constant predictor after that initial transient.

The dynamics in the spatial latent space are trivially stable. The model doesn't diverge, but it completely loses temporal resolution. The predicted trajectory collapses to a time-independent approximation of the solution. That's fine for the slowly evolving heat equation, but it would fail miserably for oscillatory dynamics like the wave equation.

5.4 Spatial versus Spectral Representation

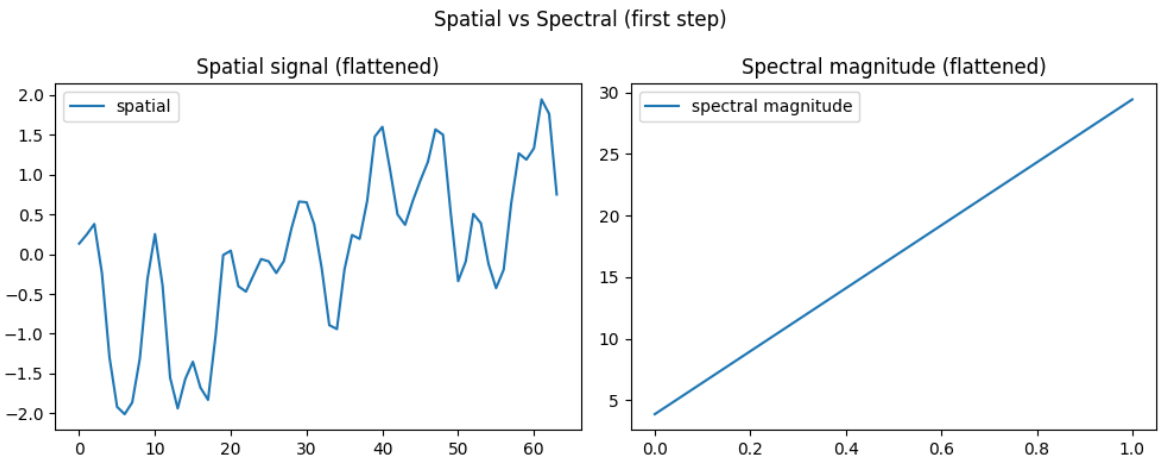

Figure 4: Comparison of spatial and spectral (Fourier magnitude) representations of the same initial condition. The spatial signal is oscillatory and non-smooth, while the spectral magnitude is monotonically increasing and structurally simpler.

Figure 4 puts spatial and spectral representations side by side for the same PDE snapshot. On the left, you see the spatial field with all its complex oscillatory structure and multiple local extrema. It's a high-entropy signal that's genuinely hard to compress without losing important information.

On the right is the corresponding Fourier magnitude spectrum. It's strikingly smooth and monotonically increasing. This smoothness comes directly from how we set up the initial condition's spectral content. The energy distributes across modes with such a regular structure that the autoencoder can really exploit it for efficient compression.

This contrast highlights why the spectral approach has such an advantage. Compressing that smooth, structured spectral representation needs fewer latent dimensions to achieve the same reconstruction quality. The resulting latent trajectories are smoother too, which makes them easier for the dynamics model to learn. The spatial autoencoder, on the other hand, has to waste representational capacity on capturing fine-grained oscillations instead of focusing on dynamically relevant features.

5.5 Noise Robustness

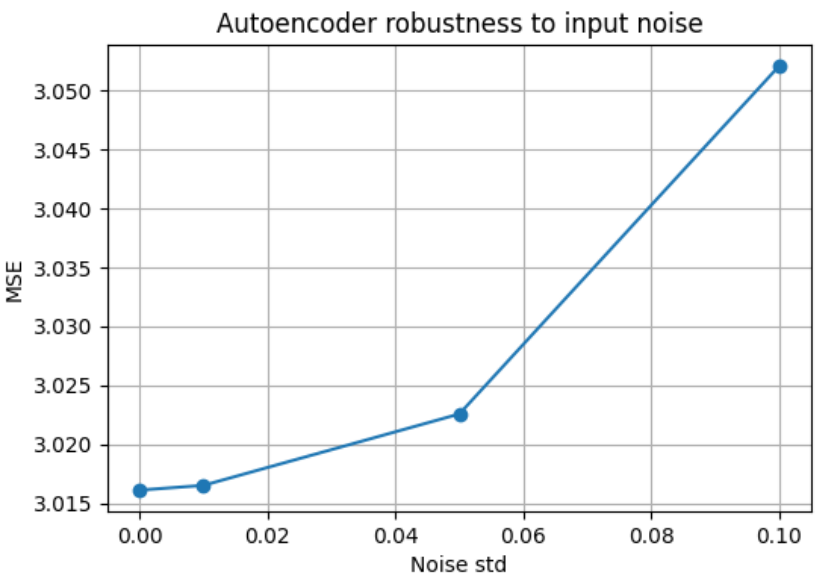

Figure 5: Autoencoder reconstruction MSE as a function of input noise standard deviation. The model degrades gracefully, with MSE increasing from about 3.016 to about 3.052 as σ goes from 0 to 0.1.

Figure 5 shows how well the autoencoder handles noisy inputs. We added Gaussian perturbations with standard deviations $\sigma \in \{0, 0.01, 0.05, 0.1\}$ to the spectral input at test time and measured reconstruction MSE.

The model degrades gracefully. MSE goes from about 3.016 with clean inputs to 3.052 at $\sigma = 0.1$. That's only a 1.2% relative increase across a pretty substantial noise range. Two factors explain this robustness.

First, the autoencoder's bottleneck acts as a regularizer. It projects noisy inputs onto the learned latent manifold, which effectively denoises them. Second, the spectral representation concentrates signal energy in low-frequency modes, while additive Gaussian noise spreads its energy uniformly across all frequencies. The autoencoder has learned to preserve that low-frequency signal and discard high-frequency content, which naturally filters out a big chunk of the noise energy.

That baseline MSE of about 3.016 even for clean inputs tells us the autoencoder has non-trivial reconstruction error. That's the trade-off you accept with low-dimensional latent representations. But the error staying so stable under perturbation is encouraging for real-world deployment where input data might be noisy or corrupted.

5.6 Comparative Summary

Table 1: Summary of rollout error characteristics across the three approaches.

Method	**MSE at Step 10**	**MSE at Step 20**	**Growth Pattern**
FNO (autoregressive)	~ 0.45	~ 1.57	Super-linear
Spectral Latent Dynamics	~ 0.37	~ 0.51	Sub-linear (saturating)
Spatial AE + Dynamics	~ 0.27	~ 0.27	Plateau (after transient)

Table 1 pulls everything together. The spectral latent dynamics model hits the sweet spot between short-horizon accuracy and long-horizon stability. It avoids the FNO's unbounded error growth while keeping meaningful temporal evolution, unlike the spatial autoencoder which just collapses to a stationary prediction. At step 10, the FNO and spectral latent model have comparable error, but by step 20 the FNO's error is three times larger.

6 Discussion

Why spectral models perform differently. The spectral representation diagonalizes the dynamics of linear PDEs. It turns a coupled spatial system into independent modal equations. When you compress this through an autoencoder, the resulting latent space inherits that decoupled structure. Temporal prediction becomes much more tractable. The dynamics model only needs to learn smooth, monotonic transformations of a few dominant modes instead of complex spatial interactions. This explains both the sub-linear error growth and the saturation behavior we saw.

When FNO outperforms latent models. The FNO actually does better in the short term, with lower error for $t < 5$. It captures single-step dynamics more accurately than our encode-predict-decode pipeline. If you only need a few prediction steps, the FNO is still competitive. Its main weakness shows up in long rollouts, where compounding errors take over.

Limitations. We only looked at linear PDEs with periodic or simple boundary conditions. Nonlinear systems like Burgers' equation or Navier-Stokes introduce energy transfer across scales, which might undermine the spectral approach's advantages. The autoencoder's fixed reconstruction error floor (about 3.0 MSE in the noise experiments) isn't negligible. More expressive architectures or variational formulations might reduce it. Also, our MLP dynamics model doesn't use any temporal context. Recurrent or attention-based alternatives could improve multi-step accuracy.

Generalization considerations. Everything was trained and evaluated on synthetic data generated with the same PDE parameters and initial condition distributions. We haven't tested how well these models generalize to out-of-distribution initial conditions, different PDE coefficients, or unseen PDE types. Transfer learning and meta-learning strategies might help address these limitations in future work.

7 Conclusion

We introduced Spectral Latent Dynamics, a framework for learning how PDE solutions evolve over time in a Fourier-compressed latent space. By transforming spatial snapshots to their spectral representation before autoencoder compression, we get latent spaces with smoother structure and more predictable dynamics than spatial alternatives.

Our main finding is that the spectral latent dynamics model achieves bounded error growth during multi-step rollouts, saturating at about 0.5 MSE after 15 steps. In contrast, the Fourier Neural Operator shows super-linear error accumulation, hitting 1.57 MSE over the same horizon. The spatial autoencoder, while trivially stable, collapses to a time-independent prediction that completely misses temporal evolution.

The core insight here is that the choice of representation space for latent compression fundamentally determines how learnable and stable the resulting dynamics model will be. Spectral representations, with their energy compaction and decoupling properties, provide a natural foundation for learning PDE dynamics in reduced-order spaces.

Next, we plan to extend this framework to nonlinear PDEs like Burgers' equation and the incompressible Navier-Stokes equations, where energy transfer between scales introduces new challenges. We also want to explore higher-dimensional domains, adaptive spectral truncation strategies, and physics-informed loss terms to further regularize the latent dynamics. Recurrent and transformer-based dynamics models might improve long-horizon accuracy by incorporating temporal context beyond the single-step Markov assumption.

A Pseudocode

Algorithm 1: PDE Data Generation (1D Heat Equation)

Input: Grid size N, time steps T, diffusivity α, number of samples N_s

Output: Dataset $\mathcal{D} = \{(\mathbf{u}_0^{(i)}, \ldots, \mathbf{u}_{T-1}^{(i)})\}_{i=1}^{N_s}$

$\Delta x \leftarrow 1/(N-1)$;

$\Delta t \leftarrow 0.4 \cdot (\Delta x)^2/\alpha$; `// CFL condition`

for $i = 1$ **to** N_s **do**

  Sample coefficients $a_m \sim \mathcal{N}(0, 1/m^2)$ for $m = 1, \ldots, M$;

  $u_0^{(i)}(x_j) \leftarrow \sum_m a_m \sin(m\pi x_j)$;

  for $n = 0$ **to** $T-2$ **do**

    $u_{n+1}^{(i)}(x_j) \leftarrow u_n^{(i)}(x_j) + \alpha \frac{\Delta t}{(\Delta x)^2}\big(u_n^{(i)}(x_{j+1}) - 2u_n^{(i)}(x_j) + u_n^{(i)}(x_{j-1})\big)$;

  end

  Append trajectory to $\mathcal{D}$;

end

Algorithm 2: Spectral Latent Dynamics Training

Input: Dataset $\mathcal{D}$, latent dim d, epochs E_1, E_2

`// Stage 1: Train Autoencoder`

for *epoch* $= 1$ **to** E_1 **do**
  foreach *batch* $\{\mathbf{u}_t\}$ *from* $\mathcal{D}$ **do**
    $\tilde{\mathbf{u}}_t \leftarrow [\mathrm{Re}(\mathrm{FFT}(\mathbf{u}_t)),\ \mathrm{Im}(\mathrm{FFT}(\mathbf{u}_t))]$;
    $\hat{\tilde{\mathbf{u}}}_t \leftarrow D_\theta(E_\phi(\tilde{\mathbf{u}}_t))$;
    $\mathcal{L} \leftarrow \|\hat{\tilde{\mathbf{u}}}_t - \tilde{\mathbf{u}}_t\|_2^2$;
    Update ϕ, θ via Adam;
  end
end

Freeze ϕ, θ;

`// Stage 2: Encode all trajectories`

$\mathcal{D}_z \leftarrow \{(\mathbf{z}_t, \mathbf{z}_{t+1}) : \mathbf{z}_t = E_\phi(\tilde{\mathbf{u}}_t)\}$;

`// Stage 3: Train Dynamics Model`

for *epoch* $= 1$ **to** E_2 **do**
  foreach *batch* $\{(\mathbf{z}_t, \mathbf{z}_{t+1})\}$ *from* $\mathcal{D}_z$ **do**
    $\hat{\mathbf{z}}_{t+1} \leftarrow f_\psi(\mathbf{z}_t)$;
    $\mathcal{L} \leftarrow \|\hat{\mathbf{z}}_{t+1} - \mathbf{z}_{t+1}\|_2^2$;
    Update ψ via Adam;
  end
end

References

[1] M. Raissi, P. Perdikaris, and G. E. Karniadakis, "Physics-informed neural networks: A deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations," *Journal of Computational Physics*, vol. 378, pp. 686–707, 2019.

[2] Z. Li, N. Kovachki, K. Azizzadenesheli, B. Liu, K. Bhatt, A. Stuart, and A. Anandkumar, "Fourier neural operator for parametric partial differential equations," in *International Conference on Learning Representations*, 2021.

[3] L. Lu, P. Jin, G. Pang, Z. Zhang, and G. E. Karniadakis, "Learning nonlinear operators via DeepONet based on the universal approximation theorem of operators," *Nature Machine Intelligence*, vol. 3, no. 3, pp. 218–229, 2021.

[4] S. L. Brunton and J. N. Kutz, *Data-Driven Science and Engineering: Machine Learning, Dynamical Systems, and Control*. Cambridge University Press, 2019.

[5] D. Kochkov, J. A. Smith, A. Alieva, Q. Wang, M. P. Brenner, and S. Hoyer, "Machine learning–accelerated computational fluid dynamics," *Proceedings of the National Academy of Sciences*, vol. 118, no. 21, 2021.

[6] N. Geneva and N. Zabaras, "Transformers for modeling physical systems," *Neural Networks*, vol. 146, pp. 272–289, 2022.

[7] B. Lusch, J. N. Kutz, and S. L. Brunton, "Deep learning for universal linear embeddings of nonlinear dynamics," *Nature Communications*, vol. 9, no. 1, p. 4950, 2018.

[8] K. Champion, B. Lusch, J. N. Kutz, and S. L. Brunton, "Data-driven discovery of coordinates and governing equations," *Proceedings of the National Academy of Sciences*, vol. 116, no. 45, pp. 22445–22451, 2019.

[9] C. R. Gin, D. E. Shea, S. L. Brunton, and J. N. Kutz, "DeepGreen: deep learning of Green's functions for nonlinear boundary value problems," *Scientific Reports*, vol. 11, p. 21614, 2021.

[10] N. Kovachki, Z. Li, B. Liu, K. Azizzadenesheli, K. Bhatt, A. Stuart, and A. Anandkumar, "Neural operator: Learning maps between function spaces with applications to PDEs," *Journal of Machine Learning Research*, vol. 24, no. 89, pp. 1–97, 2023.